

COMP 10066

Week 8 Lecture

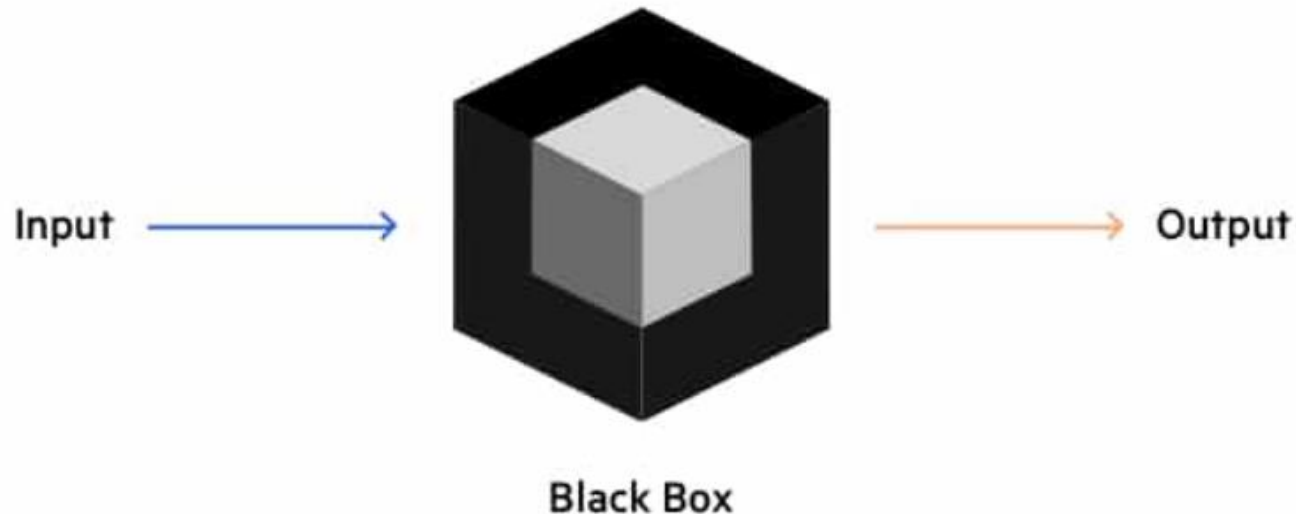
Exploratory and Black Box Testing

Exploratory Testing

- Exploratory testing is often thought of as a black box testing technique
- Instead, those who have studied it consider it a test approach that can be applied to any test technique, at any stage in the development process.
- The key is not the test technique nor the item being tested or reviewed; **the key is the active engagement of the tester**, and the tester's responsibility for managing his or her time.

Black Box Testing

- Testing that is done where access to the program source code is not available and/or needed.



Black Box Testing

- The test technique is executed by applying an input that will produce a known response that can be evaluated for correctness.
 - Correctness can be evaluated against either a known (expected) response/result that has been specified in the requirements
 - OR
 - Confirmed against a known standard response/result (Test Oracle).
 - Independent/External validation

Black Box Test Case

- Each Test Case Requires
 - A series of data to be used in the test (input)
 - A reason for the test (what is the main goal)
 - The expected result(s)
 - Based on input you must be able to predict the output based on the information provided for the application.
 - The actual result(s) – measurement
 - Test Case Outcome / Result

Test Case Outcome / Result

- A test case has a successful or not successful outcome
- Successful Test Case
 - Expected Result matches the Actual Result
- Not Successful Test Case
 - Expected Result does not match the Actual Result
 - Need to investigate and confirm issue
 - Leads to more test cases
 - Once issue is confirmed, then a bug would need to be raised

Lecture Handout

Test to Pass

- Normal data that a user would enter into an application.
- User is using the program as expected
 - Not in a malicious manner or attempt to break it
- It is expected that the program will accept AND produce the correct output based on Test to Pass data.

Test to Fail

- Data sets that are not acceptable.
- User is using the program in a malicious manner (not as expected)
- It is expected that the program will *NOT accept or use the data and that it will not impact prior calculated results or impact the program in other ways.*
- An error message or correct guidance is expected

Normal Data Definition

- For domain specific applications, the user has knowledge or has been provided with this knowledge.
- Example: Postal code in Canada vs Zip code in U.S.
 - Application may guide user on normal data through instructions or examples
- Can you come up with a test to pass and test to fail test case for a Canadian postal code?

Postal Code Example

- L8M 3H1
 - An alternating pattern of letters and numbers with a space after the first three letters. Should be accepted
- L8S3G5
 - An alternating pattern with no space between. Should be accepted.
- 3K5H7H
 - An invalid postal code but still normal data. Should produce an error message
- L8M3H, L8M3H1e
 - An invalid postal code (not long enough, too long), Should produce an error message

Test Case Outcome / Result

- Both Test to Pass and Test to Fail will have an outcome (Successful or Not Successful)
- For example a test to fail test case that does not produce an error message when required would be Not Successful

Test Cases

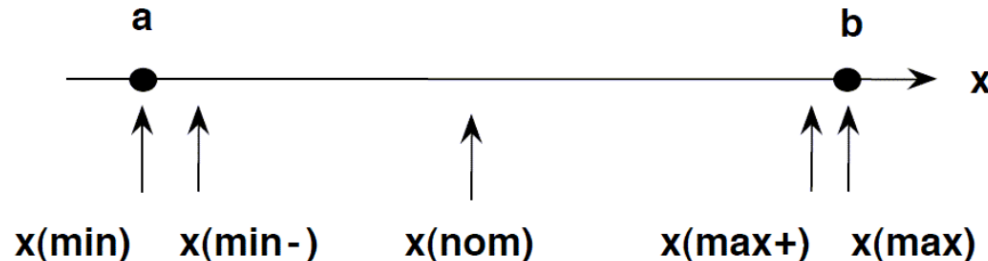
- Each Test Data set (input) will form the basis of a single Test Case (single reason for test)
- Things to think about when creating test cases
 - How many Test Cases do we need?
 - When do we stop testing?
 - What do we emphasize?
 - Test to Pass
 - Test to Fail

Boundary Testing

- It is not feasible to test every single input value – complete data coverage
- Bugs are more likely to occur at the bounds of a input value

Boundary Testing

- Example: More likely to have a bug at the bounds of an array
 - try to access `array.length` instead of `array.length-1` to get the last element
- Therefore, when selecting values to test, it is important to test at and around the bounds

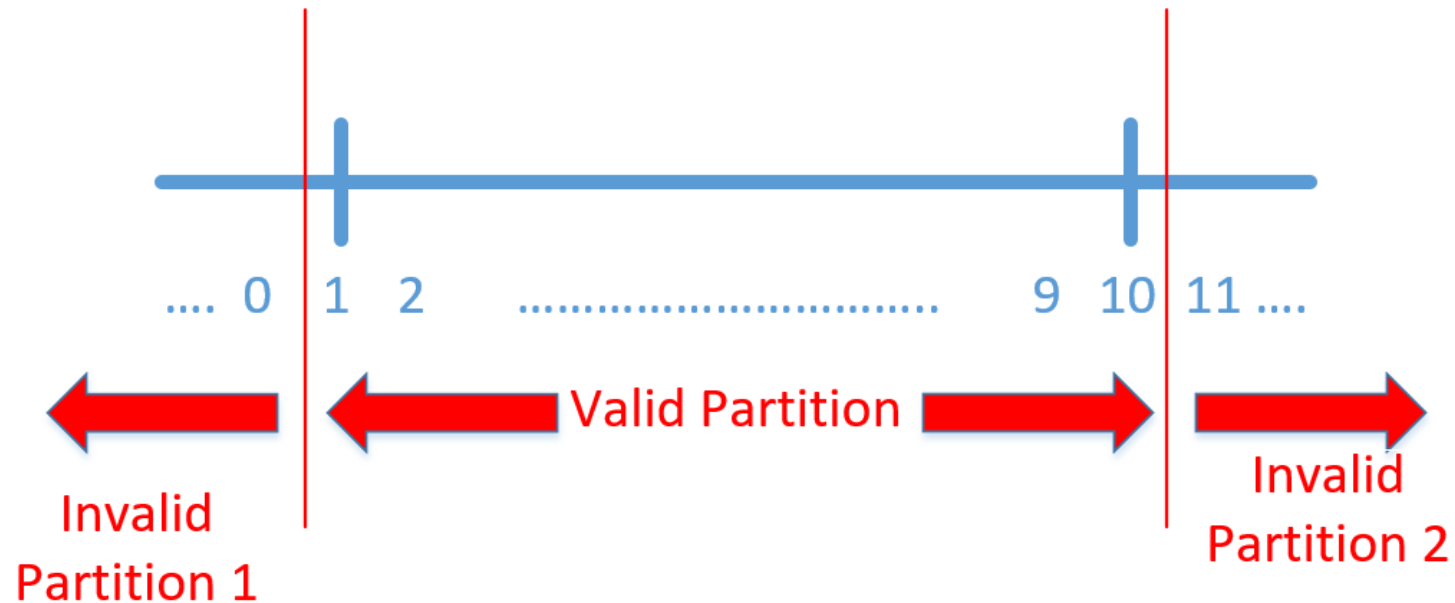


Boundary Analysis and Equivalence Partitioning

- Given a single input, which has a valid range of 1 to 10
- Determine the best values to use in testing, to ensure good data coverage
- Don't forget to think of test to pass and test to fail test cases

Boundary Analysis and Equivalence Partitioning

- Given a single input, which has a valid range of 1 to 10
- Select test data in each partition, at the boundaries



Boundary Testing Example from last slide

- Test to Pass – Valid Partition
 - 1 (lower bound)
 - 10 (upper bound)
 - 5 (a value from the middle of the partition)
- Test to Fail – Invalid Partition
 - 0 (1 below lower bound)
 - 11 (1 above upper bound)